Program 1 - Demonstrate the steps for installation of R and R Studio. Perform the following:

a) Assign different type of values to variables and display the type of variable. Assign different types such as Double, Integer, Logical, Complex and Character and understand the difference between each data type.
b) Demonstrate Arithmetic and Logical Operations with simple examples.
c) Demonstrate generation of sequences and creation of vectors.
d) Demonstrate Creation of Matrices
e) Demonstrate the Creation of Matrices from Vectors using Binding Function.
f) Demonstrate element extraction from vectors, matrices and arrays

a) Assign different type of values to variables and display the type of variable. Assign different types such as Double, Integer, Logical, Complex and Character and understand the difference between each data type.

```
# Double
   double_var <- 3.14
 print(class(double_var))
# Integer
   int_var <- 42L
 print(class(int_var))
# Logical
 logical_var <- TRUE
print(class(logical_var))
# Complex
complex_var <- 2 + 3i
print(class(complex_var))
# Character
char_var <- "Hello, World!"
print(class(char_var))
```

OUTPUT:

```
1] "numeric"

[1] "integer"

[1] "logical"

[1] "complex"

[1] "character"
```

b) Demonstrate Arithmetic and Logical Operations with simple examples.

```
# Arithmetic operations

a <- 5

b <- 3

sum_ab <- a + b

diff_ab <- a - b

product_ab <- a * b

quotient_ab <- a / b

# Logical operations

x <- TRUE

y <- FALSE

# Logical AND

and_result <- x && y

# Logical OR

or_result <- x || y

# Logical NOT

not_x <- !x

print(paste("Sum: ", sum_ab))

print(paste("Difference: ", diff_ab))

print(paste("Product: ", product_ab))
```

```
print(paste("Quotient: ", quotient_ab))
print(paste("AND Result: ", and_result))
print(paste(" OR Result:  ", or_result))
print(paste("NOT x: ", not_x))
```

OUTPUT

```
[1] "Sum:  8"
[1] "Difference:  2"
[1] "Product:  15"
[1] "Quotient:  1.66666666666667"
[1] "AND Result:  FALSE"
[1] " OR Result:   TRUE"
[1] "NOT x:  FALSE"
```

c) Demonstrate generation of sequences and creation of vectors.

1. Using the colon operator (:)

```
# Creates a sequence from 1 to 10
seq_colon <- 1:10
print("Sequence using colon operator: ")
print(seq_colon)
```

OUTPUT

```
[1] "Sequence using colon operator: "
 [1]  1  2  3  4  5  6  7  8  9 10
```

2. Using seq() function

```
# seq(from, to, by) - specify start, end, and step
seq_by <- seq(from = 1, to = 20, by = 2)
print("Sequence using seq() with step 2: ")
print(seq_by)
```

OUTPUT

```
[1] "Sequence using seq() with step 2: "
 [1]  1  3  5  7  9 11 13 15 17 19
```

```
# seq(from, to, length.out) - specify number of elements
seq_length <- seq(from = 0, to = 1, length.out = 5)
print("Sequence using seq() with fixed length: ")
print(seq_length)
```

OUTPUT

```
[1] "Sequence using seq() with fixed length: "
[1] 0.00 0.25 0.50 0.75 1.00
```

3. Using sequence() function

```
# sequence(n) generates 1:n
seq_func <- sequence(5)
print("Sequence using sequence() function: ")
print(seq_func)
```

OUTPUT

```
[1] "Sequence using sequence() function: "
[1] 1 2 3 4 5
```

4. Creating vectors directly with c()

```
vec_direct <- c(2, 4, 6, 8, 10)
print("Vector created directly with c(): ")
print(vec_direct)
```

OUTPUT

```
[1] "Vector created directly with c(): "
[1]  2  4  6  8 10
```

5. Combining sequences into a vector

```
combined_vec <- c(seq(1, 5), seq(10, 15, by = 2))
print("Combined vector from multiple sequences: ")
print(combined_vec)
```

OUTPUT

```
[1] "Combined vector from multiple sequences: "
[1]  1  2  3  4  5 10 12 14
```

6. Repeating elements with rep()

```
rep_vec <- rep(c(1, 2, 3), times = 3)
print("Repeated vector using rep(): ")
print(rep_vec)
```

OUTPUT

```
[1] "Repeated vector using rep(): "
[1] 1 2 3 1 2 3 1 2 3
```

7. Edge case: decreasing sequence

```
dec_seq <- seq(from = 10, to = 1, by = -2)
print("Decreasing sequence: ")
print(dec_seq)
```

OUTPUT

```
[1] "Decreasing sequence: "
[1] 10  8  6  4  2
```

d) Demonstrate Creation of Matrices

1. Basic matrix creation (column-wise by default)

```
mat1 <- matrix(1:6, nrow = 2, ncol = 3)
print("Matrix 1 (column-wise): ")
print(mat1)
```

OUTPUT

```
[1] "Matrix 1 (column-wise): "
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

2. Matrix creation with row-wise filling

```
mat2 <- matrix(1:6, nrow = 2, ncol = 3, byrow = TRUE)
print("Matrix 2 (row-wise):")
print(mat2)
```

OUTPUT

```
[1] "Matrix 2 (row-wise):"
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

3. Matrix with custom row and column names

```
row_names <- c("Row1", "Row2")
col_names <- c("Col1", "Col2", "Col3")
mat3 <- matrix(10:15, nrow = 2, ncol = 3, byrow = TRUE,
               dimnames = list(row_names, col_names))
print("Matrix 3 with names:")
print(mat3)
```

OUTPUT

```
[1] "Matrix 3 with names:"
     Col1 Col2 Col3
Row1   10   11   12
```

Row2 13 14 15

e) Demonstrate the Creation of Matrices from Vectors using Binding Function.

```
# Create a vector
vec1 <- c(1, 2, 3, 4, 5, 6)
print(vec1)
```

OUTPUT

```
[1] 1 2 3 4 5 6
```

```
# Create a matrix with 2 rows and 3 columns (column-wise filling by default)
mat1 <- matrix(vec1, nrow = 2, ncol = 3)
print("Matrix 1 (column-wise):")
print(mat1)
```

OUTPUT

```
[1] "Matrix 1 (column-wise):"
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
# --- Combining multiple vectors ---
vec2 <- c(7, 8, 9, 10, 11, 12)
print(vec2)
```

OUTPUT

```
[1]  7  8  9 10 11 12
```

```
# Bind vectors as rows
vec1 <- c(1, 2, 3, 4, 5, 6)
vec2 <- c(7, 8, 9, 10, 11, 12)
mat3 <- rbind(vec1[1:3], vec2[1:3])
```

```
print("Matrix 3 (row bind):")
print(mat3)
```

OUTPUT

```
[1] "Matrix 3 (row bind):"
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    7    8    9
```

```
# Bind vectors as columns
vec1 <- c(1, 2, 3, 4, 5, 6)
vec2 <- c(7, 8, 9, 10, 11, 12)
mat4 <- cbind(vec1[1:4], vec2[1:4])
print("Matrix 4 (column bind):")
print(mat4)
```

OUTPUT

```
[1] "Matrix 4 (column bind):"
     [,1] [,2]
[1,]    1    7
[2,]    2    8
[3,]    3    9
[4,]    4   10
```

5. Edge case: Empty matrix

```
mat_empty <- matrix(numeric(0), nrow = 0, ncol = 0)
print("Empty matrix:")
print(mat_empty)
```

OUTPUT

```
[1] "Empty matrix:"
```

<0 x 0 matrix>

6. Combining vectors into a matrix using rbind and cbind

```
vec1 <- c(1, 2, 3)
vec2 <- c(4, 5, 6)
mat_rbind <- rbind(vec1, vec2)
mat_cbind <- cbind(vec1, vec2)
print("Matrix created using rbind:")
print(mat_rbind)
print("Matrix created using cbind:")
print(mat_cbind)
```

OUTPUT:

```
[1] "Matrix created using rbind:"
     [,1] [,2] [,3]
vec1    1    2    3
vec2    4    5    6
[1] "Matrix created using cbind:"
     vec1 vec2
[1,]    1    4
[2,]    2    5
[3,]    3    6
```

7. Creating a matrix from a vector with recycling

```
vec <- c(1, 2)
mat4 <- matrix(vec, nrow = 3, ncol = 3)
print("Matrix 4 (vector recycled):")
print(mat4)
```

OUTPUT

```
[1] "Matrix 4 (vector recycled):"
     [,1] [,2] [,3]
[1,]    1    2    1
[2,]    2    1    2
[3,]    1    2    1
Warning message:
In matrix(vec, nrow = 3, ncol = 3) :
  data length [2] is not a sub-multiple or multiple of the number of rows [3]
```

Below is an important Note on "Creating a matrix from a vector with recycling"

```
# Define a vector with fewer elements than needed for the matrix
vec <- c(1, 2)
# Create a 3x3 matrix from the vector
# Since vec has only 2 elements, R will recycle them to fill the matrix
mat4 <- matrix(vec, nrow = 3, ncol = 3)
# Print the result
#print("Matrix 4 (vector recycled):")
#print(mat4)
```

Output:

```
[1] "Matrix 4 (vector recycled):"
     [,1] [,2] [,3]
[1,]    1    1    1
[2,]    2    2    2
[3,]    1    1    1
```

Explanation:

vec <- c(1, 2) creates a vector with two elements.

matrix(vec, nrow = 3, ncol = 3) needs 9 elements (3×3), but only 2 are provided.

Recycling rule: R repeats (1, 2, 1, 2, 1, 2, ...) until the matrix is filled column-wise by default.

If the length of the vector does not divide evenly into the total number of elements, R will give a warning.

In this case, 2 divides 9 unevenly, so you'll see:

Warning message:

In matrix(vec, nrow = 3, ncol = 3) :

data length [2] is not a sub-multiple or multiple of the number of rows

the matrix to fill row-wise instead of column-wise, you can do:

```
vec1 <- c(1, 2, 3, 4, 5, 6, 7,8,9)

mat4_rowwise <- matrix(vec1, nrow = 3, ncol = 3, byrow = TRUE)

print(mat4_rowwise)
```

OUTPUT

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

how to avoid the recycling warning by adjusting the vector length?

when creating a matrix from a vector in R.

The key is: The vector length must be an exact multiple of the total number of elements in the matrix (nrow * ncol).

Example without warning

Vector length is a multiple of 3*3 = 9

```
vec <- c(1, 2, 3)  # length 3
```

Repeat the vector so total length is exactly 9

```
vec_expanded <- rep(vec, length.out = 9)
```

```
print(vec_expanded)
```

OUTPUT

```
[1] 1 2 3 1 2 3 1 2 3
# Create the matrix
mat4 <- matrix(vec_expanded, nrow = 3, ncol = 3)
print("Matrix 4 (no recycling warning):")
print(mat4)
 #Output:
[1] "Matrix 4 (no recycling warning):"
     [,1] [,2] [,3]
[1,]    1    1    1
[2,]    2    2    2
[3,]    3    3    3
```

f) Demonstrate element extraction from vectors, matrices and arrays

1. VECTOR EXTRACTION

```
vec <- c(10, 20, 30, 40, 50)
print(vec)
```

OUTPUT

```
[1] 10 20 30 40 50
# Single element
single_elem <- vec[3]
print(single_elem)
```

OUTPUT

```
[1] 30
# Multiple elements by position
```

```
multi_elem <- vec[c(1, 4, 5)]

print(multi_elem)
```

OUTPUT

```
[1] 10 40 50

# Range of elements

range_elem <- vec[2:4]

prin(range_elem)
```

OUTPUT

```
[1] 20  30  40

# Logical indexing

logical_elem <- vec[c(TRUE, FALSE, TRUE, FALSE, TRUE)]

print(logical_elem)
```

OUTPUT

```
[1]  10, 30, 50

# Negative indexing (exclude positions)

exclude_elem <- vec[-c(2, 5)]

print(exclude_elem)
```

OUTPUT

```
[1] 10 30 40
```

2. MATRIX EXTRACTION

```
mat <- matrix(1:9, nrow = 3, byrow = TRUE)

print(mat)

     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9
```

```
# Single element (row 2, col 3)
mat_elem <- mat[2, 3]
print(mat_elem)
OUTPUT
[1]  6
# Entire row
row2 <- mat[2, ]
print(row2)
OUTPUT
[1] 4  5  6
# Entire column
mat <- matrix(1:9, nrow = 3, byrow = TRUE)
col3 <- mat[, 3]
print(col3)
OUTPUT
[1]  3 6  9
# Submatrix
sub_mat <- mat[1:2, 2:3]
print(sub_mat)
OUTPUT
      [,1] [,2]
[1,]    2    3
[2,]    5    6
# Logical indexing on matrix
mat <- matrix(1:9, nrow = 3, byrow = TRUE)
#print(mat)
```

```
logical_mat <- mat[mat > 5]
print(logical_mat)
```

OUTPUT

```
[1] 7 8 6 9
```

3. ARRAY EXTRACTION

```
# Dimensions: 3 rows, 4 cols, 2 layers
arr <- array(1:24, dim = c(3, 4, 2))
print(arr)
```

OUTPUT

```
, , 1

     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12

, , 2

     [,1] [,2] [,3] [,4]
[1,]   13   16   19   22
[2,]   14   17   20   23
[3,]   15   18   21   24
```

```
# Single element (row 2, col 3, layer 1)
arr_elem <- arr[2, 3, 1]
print(arr_elem)
```

OUTPUT

```
[1] 8
```

```
# Entire row from layer 2
row_layer2 <- arr[1, , 2]
```

```
print(row_layer2)
OUTPUT
[1] 13 16 19 22
# Entire column from all layers
col_all_layers <- arr[, 2, ]
print(col_all_layers) # 2nd column across layers
OUTPUT
     [,1] [,2]
[1,]    4   16
[2,]    5   17
[3,]    6   18
# Subarray (rows 1-2, cols 3-4, layer 1)
sub_arr <- arr[1:2, 3:4, 1]
print(sub_arr)
OUTPUT
     [,1] [,2]
[1,]    7   10
[2,]    8   11
# OUTPUT DEMONSTRATION
cat("Vector single element:", single_elem, "\n")
cat("Vector multiple elements:", multi_elem, "\n")
cat("Vector range:", range_elem, "\n")
cat("Vector logical:", logical_elem, "\n")
cat("Vector exclude:", exclude_elem, "\n\n")

cat("Matrix element:", mat_elem, "\n")
```

```
cat("Matrix row 2:", row2, "\n")

cat("Matrix col 3:", col3, "\n")

cat("Matrix submatrix:\n"); print(sub_mat)

cat("Matrix logical (>5):", logical_mat, "\n\n")


cat("Array element:", arr_elem, "\n")

cat("Array row from layer 2:", row_layer2, "\n")

cat("Array col across layers:\n"); print(col_all_layers)

cat("Array subarray:\n"); print(sub_arr)
```

Key Points

Vectors use [] with:

Positive indices → select positions.

Negative indices → exclude positions.

Logical vectors → select TRUE positions.

Matrices use [row, col] indexing.

Arrays use [row, col, layer] indexing.

Ranges (:) and index vectors (c(...)) are common for multiple selections.

Logical conditions (mat > 5) return elements meeting criteria.

Data Frames

A data frame in R is a two-dimensional table-like structure where each column can hold different types of data, making it ideal for statistical analysis and data manipulation.

What is a Data Frame?

A **data frame** is a fundamental data structure in R used to store tabular data in rows and columns, similar to a spreadsheet or database table. Each column is a vector of equal length, but columns can be of different types, such as numeric, character, or logical. Under the hood, a data frame is essentially a **list of equal-length vectors**, with each vector representing a column

Key Points

- Each column must be of a **single data type**, but different columns can have different types.
- Data frames are versatile and widely used in R for **data analysis, modeling, and visualization**.
- They support a variety of operations like **filtering, subsetting, merging, and reshaping**, making them essential for real-world data tasks
  By understanding these basics, you can efficiently create, manipulate, and analyze data using R data frames

cat vs paste

Two commonly used functions for string manipulation in R are cat() and paste(). While both functions serve similar purposes, they exhibit distinct behaviors and are suited for different scenarios.

cat() Function

The cat() function in R is primarily used for concatenating and printing strings or objects. It concatenates its arguments and prints the result. It does not add any separators or spaces between the concatenated elements by default.

Steps for Using cat() Function

- Define the elements or vectors to concatenate.
- Use the cat() function followed by the elements or vectors as arguments.

Example

```
x <- c("Hello", "World", "!")

cat(x)
```

Output:

Hello World !

paste() Function

On the other hand, the paste() function is used for concatenating vectors after converting them to character mode. It allows for specifying a separator between the elements being concatenated.

Steps for Using paste() Function

- Define the elements or vectors to concatenate.
- Use the paste() function followed by the elements or vectors as arguments.

Example :

```
x <- c("Hello", "World", "!")

paste(x)
```

Output:

```
[1] "Hello" "World" "!"
```

Difference between cat() Function and paste() Function

Feature	cat()	paste()
Output Handling	Directly prints concatenated strings	Returns concatenated string as a single character vector
Default Separator	No separator added	Space (" ") added between elements by default
Vector Handling	Concatenates elements within a single vector	Concatenates multiple vectors simultaneously
Output Control	Less flexibility	More flexibility with additional arguments like collapse

In **R programming**, print() and cat() are both used to display output, but they behave differently and are used in different situations.

Here's a **clear comparison**:

Feature	print()
Purpose	Displays an R object in a readable form (for debugging, inspection).
Return Value	Returns the object invisibly (can be used in expressions).
Formatting	Adds quotes around strings, shows metadata for complex objects, and prints in a structured way.
Automatic Newline	Adds a newline automatically after output.
Best Use Case	Debugging, inspecting variables, printing data frames, lists, etc.

Example

```
# Using print()
x <- "Hello"
print(x)       # Output: [1] "Hello"

# Using cat()
cat(x)         # Output: Hello
cat("\n")      # Manually add newline

# Multiple values
print(c("A", "B", "C"))
# Output: [1] "A" "B" "C"

cat("A", "B", "C", "\n")
# Output: A B C
```

Program 2

Assess the Financial Statement of an Organization being supplied with 2 vectors of data: Monthly Revenue and monthly expenses for the financial year. You can create your own sample data vector for this experiment) Calculate the following financial metrics:

a. **Profit for each month.**
b. **Profit after tax for each month (Tax Rate is 30%).**
c. **Profit margin for each month equals to profit after tax divided by revenue.**
d. **Good Months – where the profit after tax was greater than the mean for the year.**
e. **Bad Months – where the profit after tax was less than the mean for the year.**
f. **The best month – where the profit after tax was max for the year.**
g. **The worst month – where the profit after tax was min for the year.**

Note:

a. **All Results need to be presented as vectors**
b. **Results for Dollar values need to be calculated with $0.01 precision, but need to be presented in Units of $1000 (i.e 1k) with no decimal points**
c. **Results for the profit margin ratio need to be presented in units of % with no decimal point.**
d. **It is okay for tax to be negative for any given month (deferred tax asset)**
e. **Generate CSV file for the data.**

In this financial statement analysis question, the terms mean:

- **Best month: The month with the highest profit after tax for the financial year.**
- **Good months: All months in which profit after tax is greater than the mean (average) monthly profit after tax for the year.**
- **Bad months: All months in which profit after tax is less than the mean monthly profit after tax.**
- **Worst month: The month with the lowest profit after tax for the financial year.**

Monthly Revenue (in Dollars)

```
revenue <- c(145000, 132000, 158000, 142000, 170000, 165000,
                        180000, 175000, 160000, 155000, 168000, 182000)

# Monthly Expenses (in Dollars)

expenses <- c(120000, 118000, 130000, 125000, 145000, 140000,
                        150000, 148000, 138000, 140000, 142000, 150000)

# Tax Rate

tax_rate <- 0.30

2a. Profit for each month

profit <- revenue - expenses

2b. Profit After Tax

profit_after_tax <- profit - (profit * tax_rate)

2c. Profit Margin

profit_margin <- (profit_after_tax / revenue) * 100

2d. Good Months

mean_profit <- mean(profit_after_tax)
good_months <- profit_after_tax > mean_profit

2e. Bad Months

bad_months <- profit_after_tax < mean_profit

2f. Best Month
```

```
best_month <- profit_after_tax == max(profit_after_tax)

2g. Worst Month
worst_month <- profit_after_tax == min(profit_after_tax)

# Convert Dollar Values into Thousands (k)

revenue_k <- round(revenue / 1000)

expenses_k <- round(expenses / 1000)

profit_k <- round(profit / 1000)

profit_after_tax_k <- round(profit_after_tax / 1000)

# Profit Margin without decimal

profit_margin_percent <- round(profit_margin)

# Display Results

cat("Revenue (k):\n")
print(revenue_k)

cat("\nExpenses (k):\n")
print(expenses_k)

cat("\nProfit (k):\n")
print(profit_k)

cat("\nProfit After Tax (k):\n")
print(profit_after_tax_k)

cat("\nProfit Margin (%):\n")
print(profit_margin_percent)
```

```
cat("\nGood Months:\n")
print(good_months)

cat("\nBad Months:\n")
print(bad_months)

cat("\nBest Month:\n")
print(best_month)

cat("\nWorst Month:\n")
print(worst_month)

# Create Data Frame

financial_data <- data.frame( Revenue = revenue, Expenses = expenses,
Profit = profit,Profit_After_Tax = round(profit_after_tax, 2), Profit_Margin =
round(profit_margin, 2), Good_Month = good_months,
Bad_Month = bad_months, Best_Month = best_month, Worst_Month =
worst_month
)

# Display Data Frame

print(financial_data)

# Export Data to CSV

write.csv(financial_data,file = "Financial_Statement.csv", row.names = FALSE)

cat("\nCSV file 'Financial_Statement.csv' has been created successfully.\n")
```

OUTPUT

```
Revenue (k):
 [1] 145 132 158 142 170 165 180 175 160 155 168 182
Expenses (k):
```

[1] 120 118 130 125 145 140 150 148 138 140 142 150

Profit (k):

[1] 25 14 28 17 25 25 30 27 22 15 26 32

Profit After Tax (k):

[1] 18 10 20 12 18 18 21 19 15 10 18 22

Profit Margin (%):

[1] 12 7 12 8 10 11 12 11 10 7 11 12

Good Months:

[1] TRUE FALSE TRUE FALSE TRUE TRUE TRUE TRUE FALSE FALSE TRUE TRUE

Bad Months:

[1] FALSE TRUE FALSE TRUE FALSE FALSE FALSE FALSE TRUE TRUE FALSE FALSE

Best Month:

[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE

Worst Month:

[1] FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE

	Revenue	Expenses	Profit	Profit_After_Tax	Profit_Margin	Good_Month	Bad_Month
1	145000	120000	25000	17500	12.07	TRUE	FALSE
2	132000	118000	14000	9800	7.42	FALSE	TRUE
3	158000	130000	28000	19600	12.41	TRUE	FALSE
4	142000	125000	17000	11900	8.38	FALSE	TRUE
5	170000	145000	25000	17500	10.29	TRUE	FALSE
6	165000	140000	25000	17500	10.61	TRUE	FALSE
7	180000	150000	30000	21000	11.67	TRUE	FALSE
8	175000	148000	27000	18900	10.80	TRUE	FALSE
9	160000	138000	22000	15400	9.62	FALSE	TRUE

```
10  155000   140000    15000         10500                  6.77          FALSE          TRUE
11  168000   142000    26000         18200                 10.83          TRUE           FALSE
12  182000   150000    32000         22400                 12.31          TRUE           FALSE
   Best_Month          Worst_Month
1       FALSE               FALSE
2       FALSE               TRUE
3       FALSE               FALSE
4       FALSE               FALSE
5       FALSE               FALSE
6       FALSE               FALSE
7       FALSE               FALSE
8       FALSE               FALSE
9       FALSE               FALSE
10      FALSE               FALSE
11      FALSE               FALSE
12       TRUE               FALSE
CSV file 'Financial_Statement.csv' has been created successfully.
```

OR

```
# Financial Statement Analysis

# Monthly Revenue (in Dollars)
revenue <- c(145000, 132000, 158000, 142000, 170000, 165000,
             180000, 175000, 160000, 155000, 168000, 182000)
```

```
# Monthly Expenses (in Dollars)
expenses <- c(120000, 118000, 130000, 125000, 145000, 140000,
              150000, 148000, 138000, 140000, 142000, 150000)

# Months
months <- month.name

# Tax rate
tax_rate <- 0.30

# a. Calculate Profit
profit <- revenue - expenses

# b. Calculate Profit After Tax
tax <- profit * tax_rate
profit_after_tax <- profit - tax

# c. Calculate Profit Margin (%)
profit_margin <- (profit_after_tax / revenue) * 100

# Mean Profit After Tax
mean_pat <- mean(profit_after_tax)
```

```
# d. Good Months
good_months <- months[profit_after_tax > mean_pat]

# e. Bad Months
bad_months <- months[profit_after_tax < mean_pat]

# f. Best Month
best_month <- months[which.max(profit_after_tax)]

# g. Worst Month
worst_month <- months[which.min(profit_after_tax)]

# Convert dollar values to units of $1000
# Round to nearest whole thousand for presentation
revenue_k <- round(revenue / 1000)
expenses_k <- round(expenses / 1000)
profit_k <- round(profit / 1000)
tax_k <- round(tax / 1000)
pat_k <- round(profit_after_tax / 1000)

# Round profit margin to whole percentage
margin_percent <- round(profit_margin)
```

```
# Display results as vectors
cat("Revenue ($1000):\n")
print(revenue_k)

cat("Expenses ($1000):\n")
print(expenses_k)

cat("Profit ($1000):\n")
print(profit_k)

cat("Tax ($1000):\n")
print(tax_k)

cat("Profit After Tax ($1000):\n")
print(pat_k)

cat("Profit Margin (%):\n")
print(margin_percent)

cat("Mean Profit After Tax ($1000):\n")
print(round(mean_pat / 1000))
```

```
cat("Good Months:\n")
print(good_months)

cat("Bad Months:\n")
print(bad_months)

cat("Best Month:\n")
print(best_month)

cat("Worst Month:\n")
print(worst_month)

# Generate CSV file
financial_data <- data.frame(
  Month = months,
  Revenue = revenue,
  Expenses = expenses,
  Profit = round(profit, 2),
  Tax = round(tax, 2),
  Profit_After_Tax = round(profit_after_tax, 2),
  Profit_Margin = round(profit_margin, 2)
)
```

```
write.csv(financial_data, "financial_statement.csv",
          row.names = FALSE)

cat("CSV file generated: financial_statement.csv\n")
```

OUTPUT

```
Revenue ($1000):
 [1] 145 132 158 142 170 165 180 175 160 155 168 182
Expenses ($1000):
 [1] 120 118 130 125 145 140 150 148 138 140 142 150
Profit ($1000):
 [1] 25 14 28 17 25 25 30 27 22 15 26 32
Tax ($1000):
 [1]  8  4  8  5  8  8  9  8  7  4  8 10
Profit After Tax ($1000):
 [1] 18 10 20 12 18 18 21 19 15 10 18 22
Profit Margin (%):
 [1] 12  7 12  8 10 11 12 11 10  7 11 12
Mean Profit After Tax ($1000):
[1] 17
Good Months:
[1] "January"  "March"    "May"      "June"     "July"     "August"   "November"
"December"
```

Bad Months:

[1] "February" "April" "September" "October"

Best Month:

[1] "December"

Worst Month:

[1] "February"

CSV file generated: financial_statement.csv

Program 3: Develop a program to create two 3 X 3 matrices A and B and perform the following operations

a) **Transpose of the matrix**

b) **Addition**

c) subtraction

d) multiplication

```
# Create Matrix A
A <- matrix(1:9, nrow = 3, byrow = TRUE)
cat("Matrix A:\n")
print(A)

# Create Matrix B
B <- matrix(10:18, nrow = 3, byrow = TRUE)
cat("Matrix B:\n")
print(B)

#3a) Transpose of Matrix A

cat("Transpose of Matrix A:\n")
```

```
print(t(A))

# Transpose of Matrix B

cat("Transpose of Matrix B:\n")
print(t(B))

#3b) Addition of Matrices A and B

cat("Addition of Matrices A and B:\n")
print(A + B)

#3c) Subtraction of Matrices A and B

cat("Subtraction of Matrices A and B:\n")
print(A - B)

#3d). Multiplication of Matrices A and B (element-wise)

cat("Element-wise Multiplication of Matrices A and B:\n")
print(A * B)

#3e) Matrix Multiplication of A and B

cat("Matrix Multiplication of Matrices A and B:\n")
print(A %*% B)
```

OUTPUT

```
Matrix A:
     [,1] [,2] [,3]
```

[1,] 1 2 3

[2,] 4 5 6

[3,] 7 8 9

Matrix B:

[,1] [,2] [,3]

[1,] 10 11 12

[2,] 13 14 15

[3,] 16 17 18

Transpose of Matrix A:

[,1] [,2] [,3]

[1,] 1 4 7

[2,] 2 5 8

[3,] 3 6 9

Transpose of Matrix B:

[,1] [,2] [,3]

[1,] 10 13 16

[2,] 11 14 17

[3,] 12 15 18

Addition of Matrices A and B:

[,1] [,2] [,3]

[1,] 11 13 15

[2,] 17 19 21

[3,] 23 25 27

Subtraction of Matrices A and B:

[,1] [,2] [,3]

[1,] -9 -9 -9

[2,] -9 -9 -9

[3,] -9 -9 -9

Element-wise Multiplication of Matrices A and B:

```
     [,1] [,2] [,3]
[1,]   10   22   36
[2,]   52   70   90
[3,]  112  136  162
```

Matrix Multiplication of Matrices A and B:

```
     [,1] [,2] [,3]
[1,]   84   90   96
[2,]  201  216  231
[3,]  318  342  366
```

Program 4

Develop a program to find the factorial of given number using recursive function calls

```
#RECURSIVE FUNCTION TO FIND THE FACTORIAL OF A NUMBER
factorial <- function(n)
 { if (n==0)
   {
    return(1)
   }
   else
    {
     return(n * factorial(n-1))
    }
 }
number <- as.integer(readline(prompt = "Enter a number: "))
if(number < 0)
 {
 cat("Factorial is not defined for negative numbers. \n")
 }  else
  {
    result <- factorial(number)
```

```
  cat(paste("The factorial of", number, "is", result, "\n"))
}
```

OUTPUT

Enter a number: 5

The factorial of 5 is 120

Program 5

Develop an R Program using functions to find all the prime numbers up to a specified number by the method of Sieve of Eratosthenes

```
# Function to perform Sieve of Eratosthenes

sieve <- function(n)
 {

   # Create a logical vector and assume all numbers are prime

    prime <- rep(TRUE, n)
     # 1 is not a prime number
    if(n >= 1)
     prime[1] <- FALSE

    # Mark multiples of each prime as FALSE

    for(i in 2:floor(sqrt(n)))
    {
     if(prime[i])
       {
```

```
          for(j in seq(i * i, n, by = i))
           {
             prime[j] <- FALSE
           }
        }

     }

 return(prime)

}

# Function to display prime numbers

displayPrimes <- function(prime, n)
 {
   cat("Prime numbers up to", n, "are:\n")
    for(i in 2:n)
     {
        if(prime[i])
         cat(i, " ")

     }
}

# Main Program
n <- as.integer(readline("Enter the limit: "))
```

primeNumbers <- sieve(n)

displayPrimes(primeNumbers, n)

OUTPUT

Enter the limit: 9

Prime numbers up to 9 are:

2 3 5 7

Program 6: The built-in data set mammals contain data on body weight versus brain weight. Develop R commands to:

a) **Find the Pearson and Spearman correlation coefficients. Are they similar?**
b) **Plot the data using the plot command.**
c) **Plot the logarithm (log) of each variable and see if that makes a difference.**

```
install.packages("MASS")
library(MASS)

# Load the dataset data("mammals")

mammals_clean <- mammals[complete.cases(mammals), ]

# View the first few rows of the dataset head(mammals_clean)

print(head(mammals_clean))

#6a) Find the Pearson and Spearman correlation coefficients

# Pearson correlation

pearson_corr <- cor(mammals_clean$brain, mammals_clean$body,
method = "pearson")
cat("Pearson Correlation Coefficient:", pearson_corr, "\n")

# Spearman correlation

spearman_corr <- cor(mammals_clean$brain, mammals_clean$body,
```

```
method = "spearman")
cat("Spearman Correlation Coefficient:", spearman_corr, "\n")

#6b) Plot the data using the plot command

plot(mammals_clean$body, mammals_clean$brain, main = "Body Weight
vs. Brain Weight", xlab = "Body Weight (g)", ylab = "Brain Weight (g)", pch =
19)

#6c) Plot the logarithm (log) of each variable

log_body <- log(mammals_clean$body)

log_brain <- log(mammals_clean$brain)

plot(log_body, log_brain, main = "Logarithm of Body Weight vs. Brain
Weight", xlab = "Log of Body Weight", ylab = "Log of Brain Weight", pch =
19)
```

OUTPUT

```
WARNING: Rtools is required to build R packages but is not currently installed.
Please download and install the appropriate version of Rtools before proceeding:


https://cran.rstudio.com/bin/windows/Rtools/trying URL
'https://cran.rstudio.com/bin/windows/contrib/4.6/MASS_7.3-66.zip'

Content type 'application/zip' length 1176745 bytes (1.1 MB)

downloaded 1.1 MB


package ‘MASS’ successfully unpacked and MD5 sums checked


The downloaded binary packages are in

        C:\Users\Administrator\AppData\Local\Temp\RtmpcN7U3K\downloaded
_packages

             body brain

Arctic fox       3.385  44.5
```

Owl monkey 0.480 15.5

Mountain beaver 1.350 8.1

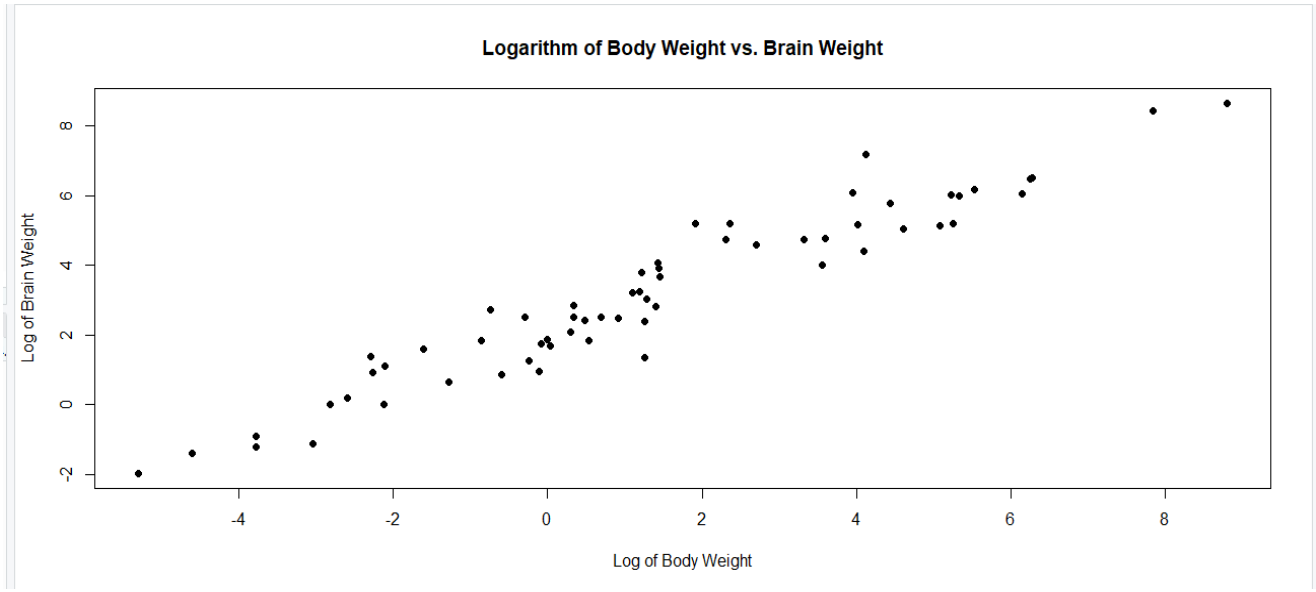
Cow 465.000 423.0

Grey wolf 36.330 119.5

Goat 27.660 115.0

Pearson Correlation Coefficient: 0.9341638

Spearman Correlation Coefficient: 0.9534986



Program 7 ; Develop R program to create a Data Frame with following details and do the following operations.

itemCode	itemCategory	itemPrice
1001	Electronics	700
1002	Desktop Supplies	300
1003	Office Supplies	350
1004	USB	400
1005	CD Drive	800

a) **Subset the Data frame and display the details of only those items whose price is greater than or equal to 350.**

b) **Subset the Data frame and display only the items**

where the category is either “Office Supplies” or “Desktop Supplies”

c) **Create another Data Frame called “item-details” with three different fields itemCode, ItemQtyonHand and ItemReorderLvl and merge the two frames**